FleetLogix — Avance #1

Proyecto Integrador | Módulo 2 Data Science | Análisis del modelo, generación y control de calidad. Documento de cierre basado en el esquema suministrado, el script 02_generate_transactions.py y los resultados SQL reportados por el estudiante.

1. Modelo relacional y restricciones

El modelo PostgreSQL contiene seis tablas, 50 columnas, seis claves primarias, cinco claves foráneas y cinco restricciones UNIQUE. Hay 21 columnas NOT NULL y 29 columnas que admiten NULL. Los campos FK del esquema original admiten NULL, aunque los datos generados fueron validados para evitar referencias huérfanas.

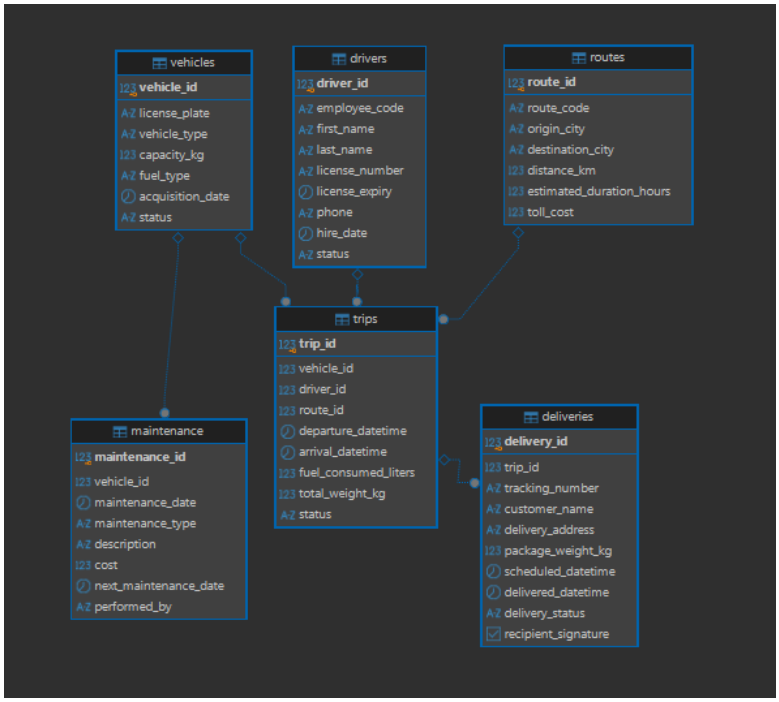
Entidad	Filas	Columnas	Propósito
vehicles	200	7	Flota
drivers	400	9	Conductores
routes	50	7	Rutas
trips	100,000	9	Viajes
deliveries	400,000	10	Entregas
maintenance	5,000	8	Servicios preventivos

Tabla dependiente	Clave foránea	Referencia
trips	vehicle_id	vehicles.vehicle_id
trips	driver_id	drivers.driver_id
trips	route_id	routes.route_id
deliveries	trip_id	trips.trip_id
maintenance	vehicle_id	vehicles.vehicle_id

Unicidad: vehicles.license_plate, drivers.employee_code, drivers.license_number, routes.route_code y deliveries.tracking_number. Valores DEFAULT: vehicles.status=active; drivers.status=active; trips.status=in_progress; deliveries.delivery_status=pending; deliveries.recipient_signature=false; routes.toll_cost=0. Los índices secundarios originales son idx_trips_departure, idx_deliveries_status e idx_vehicles_status.

2. Diagrama ER existente

Imagen suministrada por el estudiante, exportada del diseño relacional visualizado en DBeaver.



3. Patrones implícitos del negocio

Reutilización de vehículos y conductores, rutas repetidas, varios paquetes por viaje, historial de operación, mantenimiento ligado al uso, límites de capacidad, licencias vigentes y trazabilidad de entrega. Estas son reglas del negocio inferidas del enunciado y el diseño; algunas son controles Python/SQL, no constraints de PostgreSQL.

4. Metodología exacta: gethourly_distribution()

Recibe weekday con lunes=0 y domingo=6. Crea un vector NumPy de 24 pesos iniciales de 0,05; sobrescribe los pesos de las horas 06:00–21:59 para concentrar las salidas en jornada operativa. En sábado y domingo multiplica los pesos 06:00–08:59 por 0,4 y los pesos 09:00–17:59 por 0,75. Devuelve weights/weights.sum(), vector probabilístico de 24 elementos cuya suma es uno. El algoritmo modifica la distribución de HORAS elegidas, no la cantidad diaria de viajes. Las ponderaciones son hipótesis sintéticas, no datos observados de FleetLogix.

5. Metodología exacta: generate_trips()

Comprueba 200 vehículos y 50 rutas. Usa una semilla fija (20261030) y precomputa las siete distribuciones horarias. Para cada vehículo asigna un ID secuencial y dos conductores exclusivos (2i+1,2i+2). Inicia su ciudad en una de cinco ciudades y planifica 500 viajes, espaciados a lo largo de los primeros 720 días del rango [2024-10-29,2026-10-29). Toma el máximo entre la salida planificada y la llegada anterior más dos horas. Escoge una ruta cuyo origen coincide con la ciudad actual: interurbana con probabilidad de selección 3,5 % si hay rutas, local en otro caso. Calcula duración como 90–115 % de la estimada (con mínimo 0,35 h); si el viaje rebasa el final histórico intenta una alternativa local más corta. Calcula combustible con tasas base de 32, 22, 11 y 3,5 L/100 km según tipo, multiplicadas por un factor 0,87–1,18. Marca los viajes completed y actualiza ciudad y disponibilidad. Cada 20 viajes agrega un mantenimiento fechado en la llegada, con tipo cíclico, costo sintético por rango y siguiente fecha 35 días después. Regresa 100.000 viajes, un índice auxiliar por trip_id y 5.000 mantenimientos.

Los pesos de los viajes se completan después en generate_deliveries(). Esta última función asigna 2–6 entregas por viaje con distribución exacta 5/20/50/20/5, escribe 400.000 filas en flujo CSV, reparte entre 15 % y 70 % de la capacidad en paquetes usando Dirichlet y coloca las fechas de entrega dentro del viaje. Los

estados se fijan en delivered y recipient_signature=true.

6. Controles de generación y carga

validate() comprueba cantidades, rango referencial 1..N, fechas de viajes, peso positivo que no supera la capacidad, ausencia de solapamientos por vehículo/conductor, adquisiciones anteriores a mantenimientos y 25 mantenimientos por vehículo. 03_load_all.py solo carga con --load; verifica base fleetlogix_db, bloquea tablas, rechaza tablas con datos, usa COPY dentro de una transacción y ajusta las secuencias SERIAL. La carga fue confirmada por el usuario con 505.650 registros. El archivo generacion_transacciones.log deja evidencia de la generación CSV, pero no certifica por sí solo la carga.

7. Evidencias SQL reportadas y resultados

Control	Resultado reportado
Cantidad total	505.650; 100.000 trips, 400.000 deliveries, 5.000 maintenance
Integridad referencial	0 referencias inválidas de trips, deliveries y maintenance
Distribución de entregas	2:5.000; 3:20.000; 4:50.000; 5:20.000; 6:5.000
Capacidad y peso	0 sobrecargas; 0 cargas inconsistentes
Licencia y contratación	0 licencias inválidas; 0 viajes anteriores a contratación
Solapamiento	0 vehículos; 0 conductores
Uso por vehículo	500 viajes y 25 mantenimientos (razón 20:1)
Fechas viaje	Primera salida 2024-10-29 03:47; última 2026-10-18 20:52; última llegada 2026-10-19 06:15
Mantenimiento	5.000; 0 previo a adquisición; 0 próxima fecha inválida; 0 costo inválido
Tipos mantenimiento	834 aceite; 834 frenos; 833 cada una de las otras cuatro categorías

8. Mejoras propuestas sin alterar la estructura

Agregar NOT NULL a FK obligatorias; CHECK de fechas, cantidades positivas y estados permitidos; validación de capacidad entre tablas; reglas de prevención de viajes simultáneos; normalización futura de ciudades; índices de FK justificados con EXPLAIN ANALYZE. Se proponen para una evolución del modelo, no se aplican en este avance.

9. Limitaciones y alcance

La cantidad de viajes y mantenimientos es idéntica para todos los vehículos, y la frecuencia exacta no equivale a una medición operacional real. Los seis tipos de mantenimiento se alternan de forma casi uniforme. Los conductores se asignan por parejas fijas. La función horaria no reduce volumen los fines de semana. Todas las entregas se simulan como completadas y firmadas. El código trabaja con timestamps sin zona horaria y el historial real observado de primeras/últimas salidas alcanza 719,71 días, dentro de una ventana configurada de dos años. La revisión se basa en el código adjunto y los resultados SQL comunicados, sin acceso directo a la instancia PostgreSQL del estudiante.

10. Conclusión

El Avance #1 cuenta con modelo relacional analizado, diagrama ER, scripts reproducibles, 505.650 registros cargados según las verificaciones compartidas y controles principales de calidad aprobados. Para la entrega, adjuntar el log auténtico de ejecución y capturas de las validaciones en DBeaver; no incluir credenciales o .env.